INSTALLATIONS & CONFIGURATIONS USING ARGOCD

THREE WAYS TO CREATE APPLICATIONS

1) GUI
2) CLI
3) Declarative

DECLARATIVE WAY TO CREATE APPLICATIONS

create namespace
kubectl create namespace argocd

Install Argocd using below command
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

or Install

sudo kubectl apply --server-side --force-conflicts \
-n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-cd/v3.5.1/manifests/install.yaml

Expose Argocd server using port-forwarding
kubectl port-forward svc/argocd-server -n argocd 8080:443 --address=0.0.0.0 &

To get the password
kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 -d; echo

vim deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
```

```
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: my-container
        image: nginx:latest
        ports:
        - containerPort: 80
```

vim service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-svc
  labels:
    app: myapp
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - name: my-svc-port
      protocol: TCP
      port: 80
      targetPort: 80
```

vim application.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: argo-demo
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/mayurpandit25/argocd.git
    targetRevision: HEAD
    path: ./
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

1. kubectl apply -f application.yaml
2. kubectl get application -n argocd
3. kubectl get pods
4. kubectl get svc

5. Check the Argocd-Server
   http://public_ip:8080

6. check the application Sync or not
7. Then update the GitHub Repository
   1. update the deployment.yaml to replicas 4 and then commit the changes
8. Refresh the Argocd server

HOW PRUNE WORKS

1) 1.Suppose u delete the manifest file from the GitHub then argocd also delete the resource from the cluster
2) 2.delete the service manifest from the GitHub and then commit the changes
3) 3.after that refresh the argocd server u will get only the deployment file

HOW SYNC-POLICY WORKS

1) Suppose u have only the 2 pods
2) And u change the deployment state to 4
   kubectl scale deployment/my-deployment --replicas=4
3) Then when u check the pods u get only the 2 pods its because of the argocd autosync policy
4) Argocd autosync policy checks the GitHub deployment and manage these state accordingly

HOW SYN-OPTIONS WORKS

1) suppose if i want to deploy the application in the dev namespace
2) Then i have to create the namespace first
3) Instead of that i directly add the namespace in the application.yaml

```
syncOptions:
  - CreateNamespace=true
```

CREATE APPLICATIONS USING CLI

Install Argocd server

1. curl -sSL -o /usr/local/bin/argocd https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
2. chmod +x /usr/local/bin/argocd
3. argocd version --client

Login to Argo CD Server

1. argocd login localhost:8080 --username admin --password admin123 --insecure

List Argo CD Applications

1. argocd app list
2. kubectl get app -n argocd

Create Application Using CLI

```
argocd app create apache-app \
 --repo https://github.com/mayurpandit25/argocd.git \
 --path CLI-approach/apache \
 --revision main \
 --dest-server https://kubernetes.default.svc \
 --dest-namespace default \
 --sync-policy automated
```

Check Application Details

1. argocd app get apache-app

CREATE APPLICATIONS USING GUI

ON URL

http://public_ip:8080/

1. click on new app
2. application name -- nginx
3. project name ------- default
4. sync policy ---------- automatic
    1. enable sync policy
    2. enable auto heal
5. source ---------- https://github.com/mayurpandit25/argocd.git
6. revision ---------- main
7. path ---------- ./
8. destination ---------- https://kubernetes.default.svc
9. namespace ---------- default
10. create the application

11. click on application u get the pods

TO RUN ARGOCD ON LOADBALANCER

1. kubectl edit svc argocd-server -n argocd
   change: clusterIP -> LoadBalancer
2. kubectl get svc argocd-server -n argocd
